Магистратура
«Искусственный интеллект»

Инструменты разработки
Лекция 3

Чернов Евгений
НИУ ВШЭ

Git: ветвление и воркфлоу

Ветки, merge и rebase, конфликты и откат ошибок,
командная работа через Pull Request

План занятия

1. **Ветки**
   дешёвые указатели на коммиты, HEAD
2. **Слияние**
   fast-forward и трёхсторонний merge
3. **Конфликты**
   почему возникают и как их разрешать
4. **Rebase**
   перенос коммитов и золотое правило
5. **Отмена и спасение**
   cherry-pick, revert, reset, reflog
6. **Работа в команде**
   модели ветвления, Pull Request, ревью

Зачем нужны ветки

На прошлой лекции история была одной цепочкой коммитов в main. В реальном проекте одновременно идут несколько задач, эксперименты и срочные исправления — и ни одна из них не должна ломать рабочий main.

1

Изоляция

незаконченная фича или эксперимент живут в своей ветке и не трогают main

2

Параллельность

каждый работает в своей ветке, а потом изменения сводятся вместе

3

Проверка

готовую ветку можно обсудить и проверить до того, как она попадёт в main

! **Ветка — не копия папки проекта.** На диске одна рабочая копия: git switch просто подставляет файлы из другого снимка.

01

РАЗДЕЛ 01

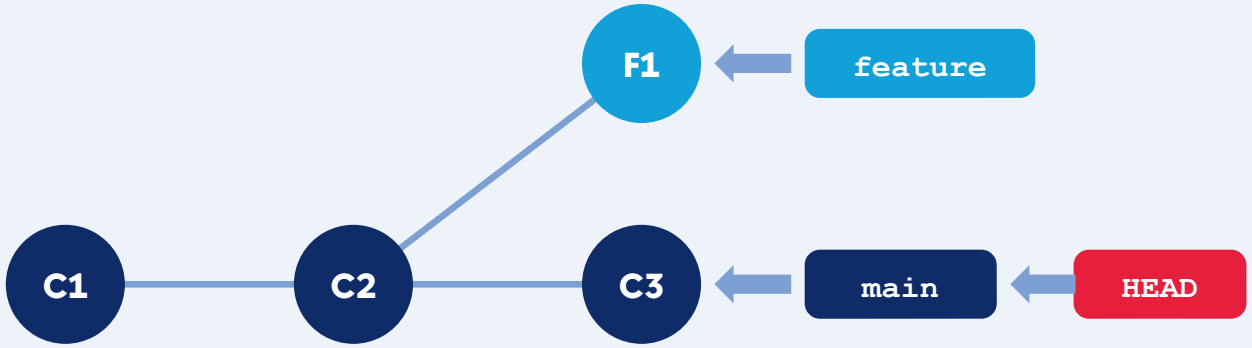
Ветки и слияние

Что такое ветка на самом деле и чем fast-forward отличается от merge-коммита

Ветка — это подвижный указатель на коммит

Ветка — просто имя, за которым хранится хеш одного коммита. HEAD отвечает на вопрос «где я сейчас» и обычно указывает на имя ветки. Новый коммит сдвигает текущую ветку вперёд.

Две ветки — один граф коммитов



HEAD → main → C3. Коммит на feature сдвинет только feature.

внутри .git

```
$ cat .git/HEAD
ref: refs/heads/main
$ cat .git/refs/heads/main
7cf494b1e0a9…
# 40 символов — хеш вершины
$ git rev-parse --short main
7cf494b
```

Создать ветку и переключиться на неё

git switch переключает ветку и пересобирает файлы на диске. Старый аналог — git checkout, он встречается в документации и ответах на форумах.

терминал

```
$ git switch -c feature/login
Switched to a new branch 'feature/login'
# правим код и коммитим
$ git commit -am "Add login form"
$ git log --oneline --graph --all
* 48caee7 (HEAD -> feature/login) Add login form
* be2a156 (main) C2: v2
* d437a5c C1: v1
$ git switch main
# на диске снова снимок main
```

`git branch`
список веток; * — текущая

`git switch <имя>`
перейти на ветку

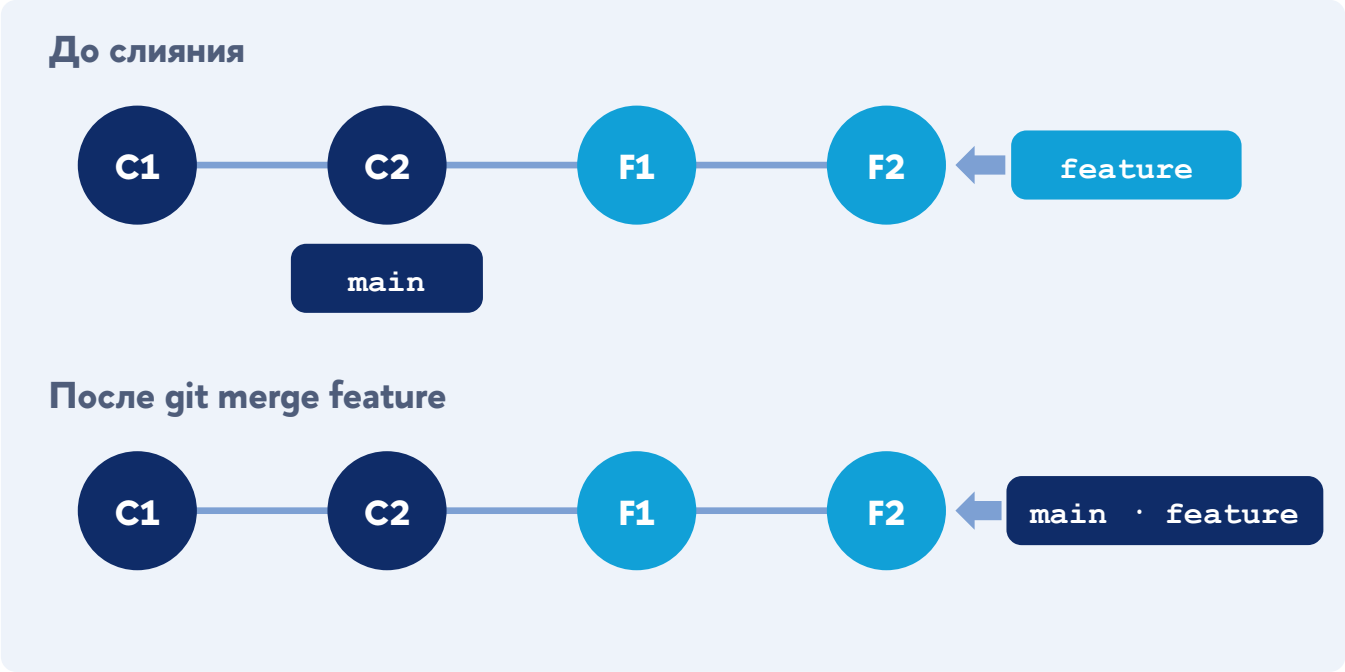
`git switch -c <имя>`
создать от текущего коммита и перейти

`git branch -d <имя>`
удалить влитую ветку; -D — силой

`git switch -`
вернуться на предыдущую ветку

Fast-forward: просто сдвинуть указатель

Если после ответвления в main не было новых коммитов, сводить нечего: Git передвигает main на вершину feature. Нового коммита нет, история остаётся линейной.



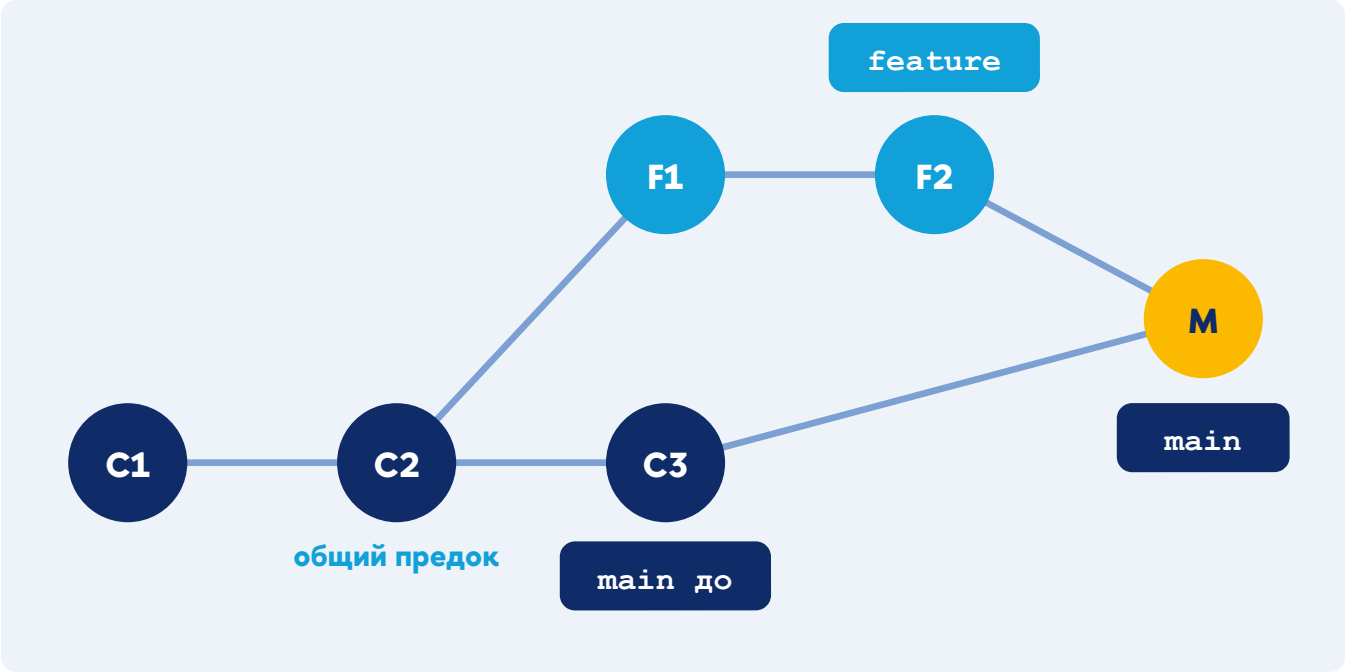
терминал

```
$ git switch main
$ git merge feature
Updating be2a156..719adca
Fast-forward
 auth.py | 1 +

# merge-коммита нет:
# main «догнала» feature
```

Трёхсторонний merge: ветки разошлись

Если новые коммиты есть и в main, и в feature, перемотать нельзя. Git сравнивает три снимка — общего предка и две вершины — и создаёт merge-коммит с двумя родителями.



терминал

```
$ git switch main
$ git merge feature
Merge made by the 'ort' strategy.
$ git cat-file -p HEAD
tree 184d2b9…
parent b28f86f…  # C3, main
parent 13aadca…  # F2, feature

# 1-й родитель — куда вливали,
# 2-й — что вливали
```

Чем закончится git merge

Ситуация	Что делает Git	Что пишет
main не двигалась	перематывает main вперёд	Fast-forward
обе ветки ушли вперёд	создаёт merge-коммит с двумя родителями	Merge made by the 'ort' strategy.
main не двигалась, флаг --no-ff	всё равно создаёт merge-коммит	Merge made by the 'ort' strategy.
ветка уже влита	ничего не делает	Already up to date.
обе изменили одни строки	останавливается и ждёт вашего решения	CONFLICT (content): …

! **merge вливает в ТЕКУЩУЮ ветку:** сначала git switch main, потом git merge feature. Сама feature при этом не меняется.

02

РАЗДЕЛ 02

Конфликты и rebase

Почему Git иногда останавливается и как перенести свои коммиты на свежий main

Почему возникает конфликт

Конфликт — не поломка, а вопрос от Git: обе ветки по-разному изменили одно и то же место файла. Правки в разных файлах или в разных частях файла Git сливает сам.

общий предок

```
port = 5000
```

main

```
port = 8080
```

feature

```
port = 3000
```

`git merge → ?`

Какую строку оставить — 8080, 3000 или третий вариант? Решать вам.

```
git merge feature

Auto-merging config.py
CONFLICT (content): Merge conflict in config.py
```

Как прочитать и разрешить конфликт

config.py во время конфликта

```
<<<<<<< HEAD
port = 8080
=======
port = 3000
>>>>>>> feature

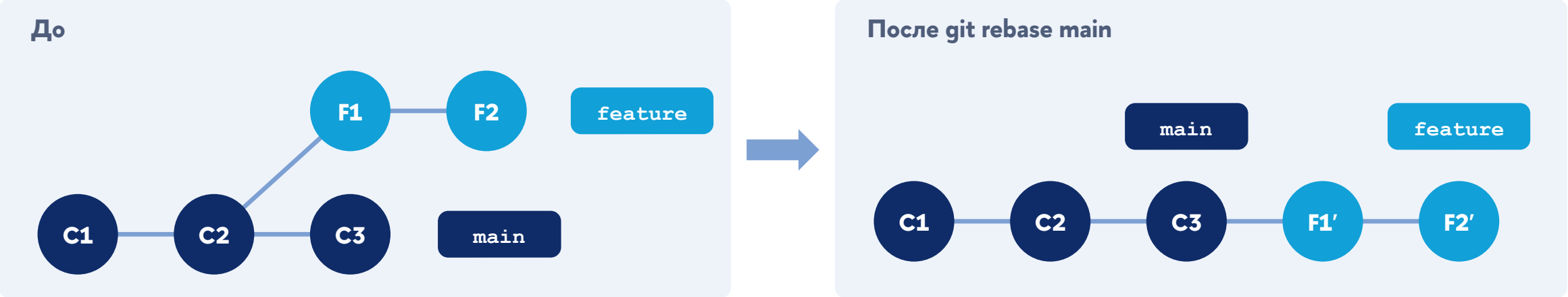
# HEAD    — текущая ветка (main)
# feature — вливаемая ветка
```

1. **`git status`** — файлы в конфликте: both modified
2. **`правка файла`** — оставить нужное, стереть маркеры
3. **`git add config.py`** — отметить: конфликт решён
4. **`git commit`** — завершить merge
5. **`git merge --abort`** — передумали: отменить merge

! **В VS Code над конфликтом есть кнопки Accept Current / Incoming / Both.** Они просто правят файл — git add и git commit всё равно нужны.

Rebase: переносим коммиты на новую базу

git rebase main по очереди применяет коммиты вашей ветки поверх свежего main. Получаются новые коммиты с тем же содержимым, но с другими хешами. Старые F1 и F2 остаются в базе, их видно в reflog.



терминал

```
$ git switch feature && git rebase main
Successfully rebased and updated refs/heads/feature.
```

Merge или rebase?

merge

- сохраняет историю такой, какой она была
- не меняет существующие коммиты
- добавляет merge-коммиты — граф ветвится
- конфликты решаются один раз, при слиянии
- безопасен для общих веток

rebase

- линейная история без merge-коммитов
- переписывает коммиты: у них новые хеши
- удобно освежить свою ветку перед PR
- конфликт может возникнуть на каждом переносимом коммите
- после rebase: git push --force-with-lease

! **Золотое правило rebase:** не переписывайте коммиты, которые уже видели другие. Свою ветку — можно; main и общие ветки — только merge или revert.

Интерактивный rebase: причёсываем историю

git rebase -i HEAD~4 открывает в редакторе последние четыре коммита — сверху самый старый. Меняете слово слева, сохраняете файл — и Git переписывает историю по этому плану.

```
git rebase -i HEAD~4

pick   1bbc191 Add login form
squash 20ac2b8 Fix typo
fixup  7cf494b Fix typo again
reword 48caee7 Add validation

# сохранить и закрыть редактор —
# Git выполнит план сверху вниз
```

pick
оставить коммит как есть

reword
оставить, но исправить сообщение

squash
слить с коммитом выше, сообщения объединить

fixup
слить с коммитом выше, сообщение выбросить

drop
удалить коммит из истории

03

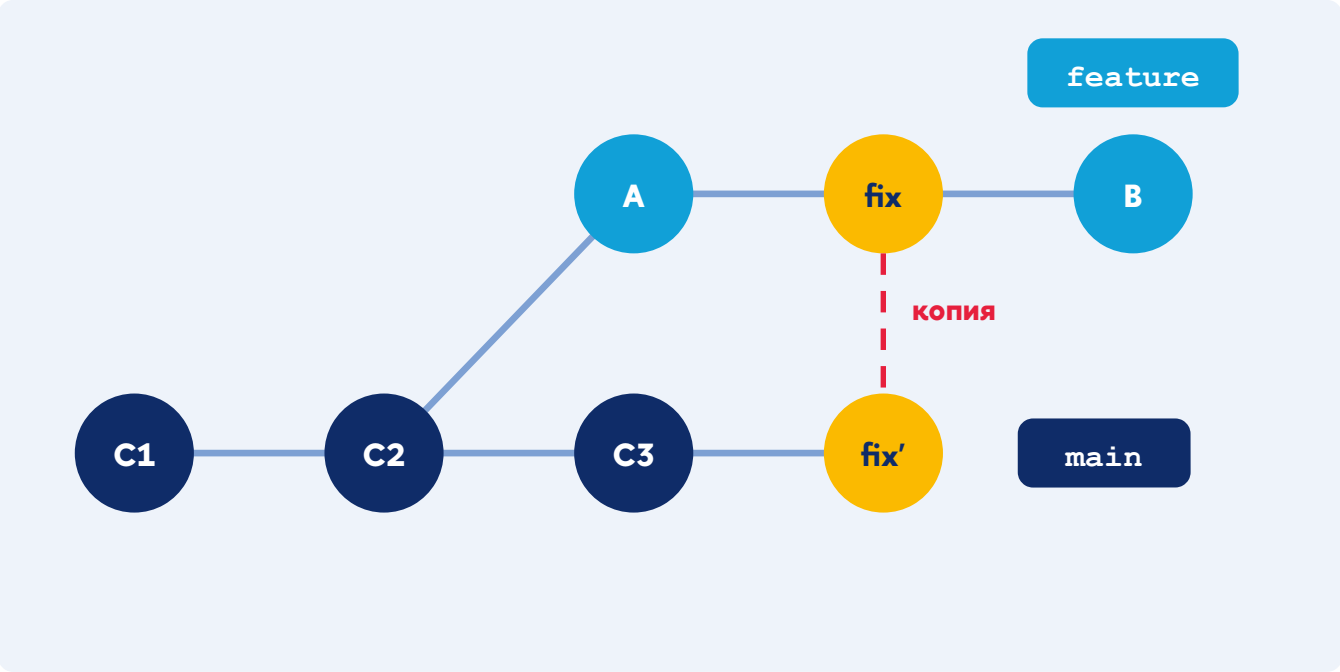
РАЗДЕЛ 03

Отмена и спасение

cherry-pick, revert, reset и reflog — как исправить ошибку и вернуть «потерянное»

cherry-pick: взять один коммит из другой ветки

Из чужой ветки нужен только один коммит — например, исправление бага, — а вливать всю ветку рано. cherry-pick применяет изменения этого коммита к текущей ветке как новый коммит.



терминал

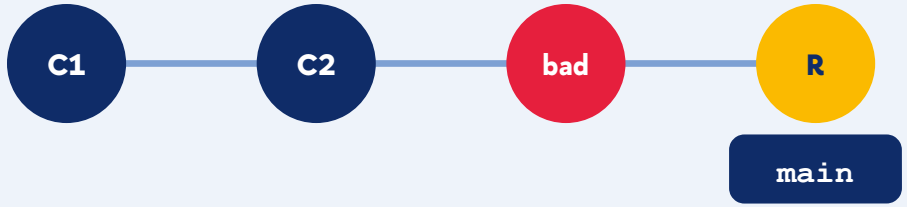
```
$ git switch main
$ git cherry-pick 1bbc191
[main 5715eed] Fix crash

# конфликт? как при merge:
# правим файл, git add, затем
$ git cherry-pick --continue
# или отменить всё:
$ git cherry-pick --abort
```

revert и reset: два способа отменить коммит

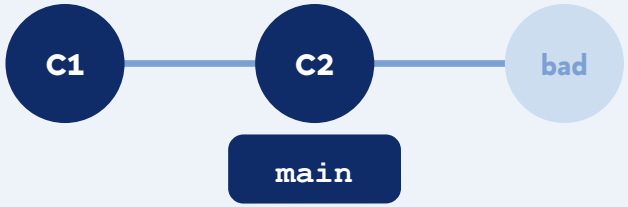
reset передвигает ветку назад — коммиты пропадают из её истории. revert ничего не стирает: он добавляет новый коммит, который отменяет изменения указанного.

`git revert <хеш>`



История цела: новый коммит R отменяет изменения bad. Годится для main и всего, что уже запушено.

`git reset --hard HEAD~1`



bad выпал из ветки: история переписана. Только для своих коммитов, которые никто не забрал.

Своё и не отправленное — reset, общее и запушенное — revert. Merge-коммит отменяют так: git revert -m 1 <хеш> (1 — первый родитель, то есть main).

reset --soft, --mixed, --hard

Все три режима передвигают ветку на указанный коммит, например HEAD~1. Разница в том, что станет с изменениями из «снятых» коммитов и с вашими незакоммиченными правками.

Команда	Индекс (staging)	Рабочая папка	Когда нужно
--soft	изменения остаются в индексе, готовы к commit	не трогает	переделать или склеить коммиты
--mixed (по умолчанию)	очищается: правки «не в индексе»	не трогает	передумали, что класть в коммит
--hard	очищается	откатывается: правки удалены	выбросить всё до состояния коммита

! **--hard стирает и незакоммиченные правки —** их не вернёт даже reflog. Закоммиченное вернуть можно — об этом следующий слайд.

reflog: спасательный круг

reflog — локальный журнал перемещений HEAD. Коммиты, пропавшие из git log после reset --hard или неудачного rebase, остаются в базе, и reflog показывает их хеши.

терминал

```
$ git reset --hard HEAD~2          # «ой!»
$ git reflog
d27f009 HEAD@{0}: reset: moving to HEAD~2
66773a9 HEAD@{1}: commit: Add report
3d0139f HEAD@{2}: commit: Add metrics
$ git reset --hard HEAD@{1}
HEAD is now at 66773a9 Add report
```

1. **`git reflog`** — найти хеш до ошибки
2. **`git reset --hard <хеш>`** — вернуть ветку туда
3. **`git switch -c rescue <хеш>`** — или поднять в новую ветку

! **reflog есть только у вас на компьютере.** Записи хранятся около 90 дней (недостижимые коммиты — 30), потом их убирает сборщик мусора.

04

РАЗДЕЛ 04

Работа в команде

Модели ветвления, Pull Request и code review — как ветки живут в реальном проекте

Модели ветвления

GitHub Flow

main всегда рабочий. Любая задача — короткая ветка от main → Pull Request → ревью → merge.

Ветки: `main, feature/*`

Когда: частые релизы, веб-сервисы, учебные проекты

Git Flow

Две долгие ветки: main — релизы, develop — интеграция. Плюс feature/*, release/*, hotfix/*.

Ветки: `main, develop + 3 типа`

Когда: релизы по расписанию, версии продукта

Trunk-based

Все вливают в main (trunk) минимум раз в день. Ветки живут часы, недоделанное прячут за флагами.

Ветки: `main + очень короткие`

Когда: сильный CI, большие команды

! **В курсе — GitHub Flow:** ветка → Pull Request → ревью → merge в main. Так же сдаются домашние задания.

Pull Request и code review

Pull Request (в GitLab — Merge Request) — запрос «влейте мою ветку в main». До слияния команда видит diff, обсуждает его, а CI прогоняет тесты.



терминал

```
$ git switch -c fix/login-crash
$ git commit -am "Fix crash on empty login"
$ git push -u origin fix/login-crash
# на GitHub: Compare & pull request,
# после ревью — Merge
```

Описание
что и зачем изменено, как проверить

Ревью
сначала логика и ошибки, потом стиль; про код, а не про человека

Защита main
прямой push запрещён, merge — после approve и CI

Сообщения Git, которые пугают новичков

Каждое из них — подсказка, а не авария. Что оно значит и что делать дальше:

`CONFLICT (content)`
Что делать: обе ветки правили одни строки — правим файл, add, commit

`Already up to date.`
Что делать: вливать нечего: ветка уже в истории текущей

`branch 'x' is not fully merged`
Что делать: ветка не влита; -D удалит имя, коммиты найдёт только reflog

`! [rejected] (fetch first)`
Что делать: на сервере есть чужие коммиты — сначала git pull, потом push

`HEAD detached at 8010b47`
Что делать: вы на коммите, а не на ветке — для работы git switch -c

`There is no merge to abort`
Что делать: merge уже закоммичен — поможет reset или revert

ИТОГИ ЛЕКЦИИ

Что важно унести

1. **Ветка — это указатель**
   дешёвое имя на коммит; HEAD — где вы сейчас
2. **merge — два исхода**
   fast-forward или merge-коммит; конфликт — вопрос, а не авария
3. **rebase — только своё**
   линейная история, но новые хеши: общую историю не трогаем
4. **Отмена без паники**
   общее — revert, локальное — reset, потерянное — reflog

Дальше в курсе — **Bash-скрипты и отладка сервера**: автоматизируем рутину и выясняем, почему сервис не отвечает.


ВЫСШАЯ • ШКОЛА • ЭКОНОМИКИ •